

1. Overview

This document traces the complete execution flow of a single research run, from the moment a user clicks 'Launch Workflow' to the moment a finished report appears in the Report Viewer. It corresponds directly to the LangGraph definition in `app/graph/workflow.py` and the trace originally documented in the project's technical walkthrough (§11).

2. Stage-by-Stage Flow

2.1 User Request

The user enters a free-text research objective into the New Research form on the dashboard and submits it. `js/dashboard.js` sends this as a `POST /api/workflows` request with the objective as JSON.

2.2 Request Analysis (Configuration Gate)

Before any agent runs, `create_workflow()` checks `openrouter_configured` and `tavily_configured`. If either is false, the request is rejected immediately with HTTP 400 and a message naming the missing key — no `WorkflowRun` row is created and no agent executes. This is the system's fail-fast boundary.

2.3 Clarification

The current implementation does not pause the workflow to ask the user a clarifying question. The objective is passed to the Supervisor as-is once configuration validation succeeds. A clarification checkpoint is part of the target reference architecture but is not implemented in this version; it is listed under Known Limitations as a gap between the reference architecture and the current build.

2.4 Research Planning (Supervisor Agent)

Once a `WorkflowRun` row is created (`status='queued'`) and the response returned to the browser, the background task builds the initial `WorkflowState` and invokes the LangGraph app. The first node, `supervisor_node`, sets `current_stage='supervisor'` and calls `SupervisorAgent.plan()`, which sends the objective to OpenRouter and receives back 3–6 specific, non-overlapping research questions as a `ResearchPlan`. This plan is logged to `execution_logs` and written into `WorkflowState`.

2.5 Task Planning

Task planning in this implementation is implicit in the Supervisor's output rather than a distinct graph node: the `ResearchPlan`'s list of research questions is itself the task list consumed directly by the Research agent. A dedicated tasks table exists in the schema for future per-question progress tracking, but it is not currently populated by the graph (see Known Limitations).

2.6 Parallel Research (Research Agent)

`research_node` calls `ResearchAgent.research_all()`, which fires a Tavily search for every research question concurrently via `asyncio.gather()` rather than looping over them sequentially. If the plan contains 4 questions, 4 searches execute at the same time. Each search returns up to 4 results (default `max_results`), and every result becomes an `EvidenceItem`.

2.7 Evidence Collection

As each Tavily result comes back, it is written to the evidence table immediately via `store_evidence()` — not batched until the end of the research stage. This means evidence already gathered survives even if a later stage of the same run fails.

2.8 Analysis (Analyst Agent)

`analyst_node` reads back every Evidence row for the current run from the database, numbers them for citation, and calls `AnalystAgent.analyze()`. The Analyst returns an `AnalysisResult` containing key insights, an optional comparison table (populated only when the topic has genuinely comparable options), trade-offs, and `unsupported_claim_warnings` for any point not well supported by the retrieved evidence.

2.9 Critic Review (Critic Agent)

`critic_node` serializes the `AnalysisResult` and calls `CriticAgent.review()`, which returns `CriticFeedback` with a verdict of `APPROVED` or `REVISION_REQUIRED`, a list of concrete issues, and a justification. Regardless of what the LLM decides, if `revision_count` has already reached `max_revisions` (default 2), the verdict is forced to `APPROVED` by a plain Python conditional in `critic.py`.

2.10 Revision Loop

`route_after_critic()` inspects the verdict: if `REVISION_REQUIRED` and the run is still under the revision cap, `revision_count` is incremented and control returns to the analyst node — with the same evidence set, plus the Critic's feedback added to context — for a fresh analysis attempt. This loop can repeat until either the Critic approves, or the hard cap is reached and approval is forced.

2.11 Report Generation (Writer Agent)

Once the Critic's verdict is `APPROVED` (naturally or via the cap), control passes to `writer_node`, which calls `WriterAgent.write()`. The Writer assembles one prompt containing the objective, research questions, all numbered evidence, and the Analyst's synthesis, and asks the LLM to produce an adaptively-structured Markdown report body citing evidence using the existing [n] numbers. After the LLM returns its body, the code deterministically appends a References section built directly from the real Evidence rows — this section cannot contain a hallucinated source because it is not LLM-generated text. The resulting `FinalReport` is saved as a new (versioned) row in the reports table.

2.12 Execution Logging

Throughout every stage above, each agent calls `BaseAgent.log_event()` to write timestamped events (started, evidence_stored, verdict, completed, etc.) to the `execution_logs` table. This log is what the Live Monitor's timeline renders, and it is also the durable audit trail read back if a run fails partway through.

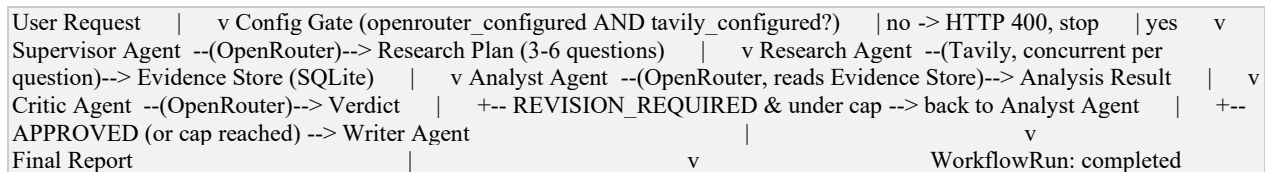
2.13 Workflow Completion

After the Writer node finishes, LangGraph routes `writer -> END`. The `WorkflowRun`'s status is set to 'completed'. On the frontend, the next poll of `GET /api/workflows/{id}` sees status: 'completed', stops the 1.5-second polling loop, shows a success toast, and automatically loads the finished report into the Report Viewer tab, rendered from Markdown to HTML via `marked.js` with a clickable table of contents generated from the report's own headings.

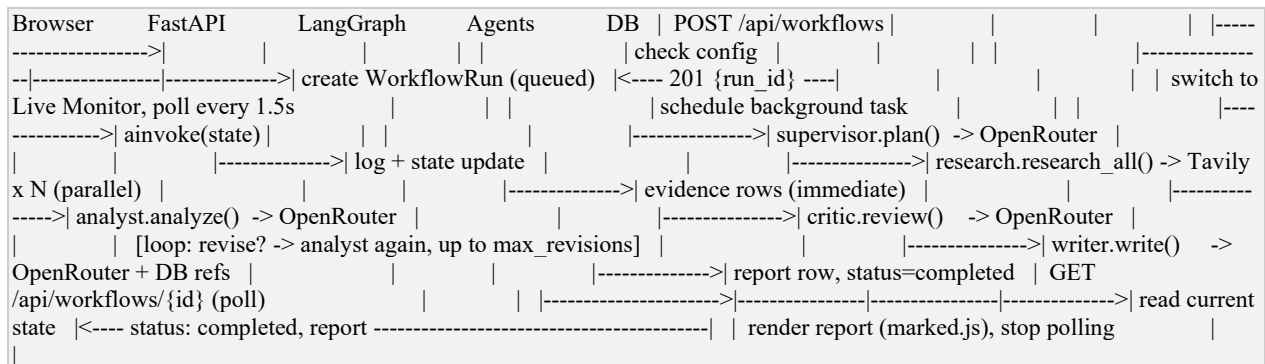
3. Failure Handling Within the Flow

If any exception occurs anywhere in steps 2.4 through 2.11 — a missing/revoked API key, an OpenRouter rate limit, JSON that fails to parse, or a network timeout that exhausts its 3 retries — it propagates up to a single outer try/except around the entire `_workflow_app.aiinvoke(...)` call in the background task. That handler marks the `WorkflowRun` as `status='failed'`, stores the real exception message in the error column, and the browser's next poll surfaces that failure to the user instead of a completed report.

4. Flowchart



5. Sequence Diagram



6. Timing Characteristics

- The API response for `POST /api/workflows` returns effectively instantly (workflow execution happens entirely after the response is sent, as a `BackgroundTask`).
- Research is the only stage with intra-stage parallelism (all research questions searched concurrently); Supervisor, Analyst, Critic, and Writer each execute as a single sequential LLM call per pass.
- Total wall-clock time is dominated by the number of OpenRouter calls, which itself depends on how many revision cycles the Critic triggers (0 to `max_revisions` additional Analyst+Critic round trips).
- The dashboard's Live Monitor polls every 1.5 seconds regardless of actual stage duration, so perceived responsiveness is bounded by that interval, not by the underlying workflow speed.